

Complex Computing Activity (CCA)

Mapped to C3: Application & Implementation of OOP in C++

Overview

Select a real-world computing problem and implement a fully functional C++ application that demonstrates an in-depth, practical application of Object-Oriented Programming principles. This activity is aligned with Course Outcome C3: students will apply OOP concepts to design, implement, and evaluate software solutions for complex computing problems. The solution must be robust, well-structured, and demonstrate innovative use of the OOP concepts covered in the CT-260 course plan.

Group Policy

Policy	Details
Group Size	Minimum 2 and maximum 3 students per group.
Individual Viva	Each group member will be questioned individually on the implementation. Marks may differ between members based on depth of knowledge demonstrated.
Code Ownership	All members must be able to explain any section of the submitted code. Plagiarism or undisclosed AI-generated code will result in zero marks for the entire group.

Technical Requirements

- OOP Concept Coverage:** The implementation must demonstrate at least three OOP pillars: Encapsulation, Inheritance, Polymorphism, Abstraction. Static members, friend functions, and Association/Aggregation/Composition must also be shown where appropriate (Weeks 1-7, 10-11).
- Templates & STL:** Incorporate at least one function template or class template (Week 12). Demonstrate use of at least one STL container (vector, stack, queue, etc.) as covered in Week 13.
- Error & Exception Handling:** The application must handle invalid inputs and edge cases gracefully using C++ exception handling mechanisms (try/catch/throw) covered in Week 13.
- File I/O:** The application must perform at least one file read or write operation (save records, load configuration, export report) as per Week 14.
- Design Pattern:** Identify and implement at least one design pattern from Week 8 (e.g., Singleton, Factory, Observer). Explain the pattern in code comments.
- Real-World Mapping:** The chosen problem must map clearly to a real-world scenario. Examples: Online Store, Game Engine, etc.
- Code Quality:** Code must be well-commented, logically organised into header (.h) and source (.cpp) files, and must compile without errors or warnings.

Deliverables

- A project folder containing all .h and .cpp source files, organised by module/class.
- A compiled executable (.exe) file that runs correctly on the evaluator's machine.
- A brief README.txt describing: how to compile and run the project, the problem domain, and a list of implemented OOP features.
- Submission due: Week 15 (Project Evaluation Week) via google classroom.

CCA Assessment Rubric | CT-260 | Total: 10 Marks | C3 - Application & Implementation

Student Name:

Roll No:

Criterion	Good (2.5)	Average (2)	Poor (1)	Marks Awarded
Criterion 1 [Max: 2.5 marks] Clarity of Objective (CA5: Familiarity) C3: Student has selected and understood a real-world problem suitable for an OOP solution in C++.	Student has complete clarity of the objective; problem is non-trivial, well-scoped, and its OOP relevance is fully justified with reference to CT-260 course concepts.	Student has some clarity; the problem is identified but the OOP mapping is partial or not sufficiently complex for a C++ implementation.	Student has misunderstood the objective; the problem is trivial, poorly defined, or does not lend itself to OOP.	/2.5
Criterion 2 [Max: 2.5 marks] Problem-Solving & OOP Design (CA3: Innovation) C3: Student has devised and designed a complete OOP solution using appropriate C++ concepts from the syllabus.	Student has designed a complete, innovative solution employing the most appropriate OOP concepts (encapsulation, inheritance, polymorphism, abstraction, templates, design patterns) with clear justification.	Student has designed an incomplete solution or has not used the most appropriate OOP concepts; some pillars are missing or misapplied.	Student has not provided a satisfactory OOP design, or has not used appropriate OOP concepts at all.	/2.5
Criterion 3 [Max: 2.5 marks] Implementation Quality (CA3: Innovation) C3: Student has implemented the C++ solution correctly and it functions robustly for varied inputs.	Solution compiles without errors or warnings, is logically correct, handles exceptions and edge cases, implements File I/O and STL, and is well-structured across .h/.cpp files.	Solution is syntactically correct but has logical errors, missing features (e.g., no exception handling or File I/O), or is not modularised properly.	Solution has not been implemented, does not compile, or contains fundamental structural errors.	/2.5
Criterion 4 [Max: 2.5 marks] Depth of Knowledge (Viva) (CA5: Familiarity) C3: Student can explain and defend all aspects of the implementation demonstrating thorough understanding.	Student answered all questions relevant to the solution confidently; demonstrated understanding of every OOP concept used and could explain design decisions clearly.	Student answered most questions; demonstrated understanding of the main OOP concepts used but struggled with design-level or edge-case questions.	Student answered few or no questions relevant to the solution; unable to explain the OOP concepts or code implemented.	/2.5
Total				/10

Teacher's Signature:

Date: